

Plan de Pruebas Funcionales

Sistema SNAAR - TekMess

Danna Andrade

Ariel Llumiquinga

David Pilaguano

9 de julio de 2026

Documento	Plan de pruebas funcionales del prototipo Tek-Mess
Versión	1.0
Fecha de elaboración	9 de julio de 2026
Proyecto	Sistema SNAAR - TekMess
Universidad	Universidad de las Fuerzas Armadas ESPE
Departamento	Departamento de Ciencias de la Computación
Asignatura	Análisis y Diseño de Software
Tema	Informe Plan de Pruebas Funcionales y Unitarias
Docente	Ing. Jenny Ruiz
Fecha	9 de julio de 2026
Tipo de pruebas	Pruebas funcionales automatizadas con JUnit
Responsables	Danna Andrade, Ariel Llumiquinga, David Pilaguano

Índice

Resumen Ejecutivo	5
Estado de Implementación	5
Requisitos Funcionales - Estado Final	5
1. Control del Documento	6
1.1. Historial de Versiones	6
1.2. Propósito del Documento	6
2. Introducción	6
3. Objetivos	6
3.1. Objetivo General	7
3.2. Objetivos Específicos	7
4. Datos Generales del Proyecto	7
5. Involucrados	8
6. Planteamiento de las Pruebas	8
6.1. Enfoque de Testing	9
6.2. Estrategia de Mocking	9
7. Alcance	9
7.1. Funcionalidades Incluidas	9
7.2. Funcionalidades No Incluidas	9
8. Estrategia de Pruebas	10
9. Herramienta	10
9.1. Framework de Testing	10
9.2. Entorno de Pruebas	11
9.3. Estructura de Archivos de Prueba	11
9.4. Estrategia de Simulación y Aislamiento	11
10. Ambiente de Pruebas	12
11. Criterios de Entrada y Salida	12

11.1. Criterios de Entrada	12
11.2. Criterios de Salida	12
12.Datos de Prueba	12
13.Matriz de Trazabilidad	13
14.Implementación por Requisito	14
14.1. REQ001: Registro de empleados y generación de credenciales	14
14.2. REQ002: Listado de personal registrado	14
14.3. REQ003: Edición de datos del empleado	15
14.4. REQ004: Eliminación de empleados	15
14.5. REQ005: Inicio de sesión por rol	16
14.6. REQ006: Cambio de contraseña	17
14.7. REQ007: Recuperación de contraseña	17
14.8. REQ008: Generación de reporte analítico	18
14.9. REQ009: Consulta de historial de reportes	18
15.Código de Implementación de las Pruebas	19
15.1. Configuración Maven para JUnit 5	19
15.2. Ajuste de Testabilidad en AuthServiceio	20
15.3. Estructura General del Archivo de Pruebas	20
15.4. Implementación de Prueba para REQ001	21
15.5. Implementación de Prueba para REQ005	21
15.6. Implementación de Prueba para REQ006	22
15.7. Implementación de Prueba para REQ008	23
15.8. Implementación de DAO en Memoria	23
15.9. Comando de Ejecución	24
16.Desglose Detallado de Pruebas Funcionales	25
16.1. TC-001: Registro correcto de empleado y generación de credenciales	25
16.2. TC-002: Rechazo de registro con cédula duplicada	26
16.3. TC-003: Listado de personal registrado	27
16.4. TC-004: Edición de datos de un empleado existente	28
16.5. TC-005: Eliminación de empleado y usuario asociado	29
16.6. TC-006: Inicio de sesión válido por rol	30

16.7. TC-007: Bloqueo de cuenta por intentos fallidos	31
16.8. TC-008: Cambio de contraseña válido	32
16.9. TC-009: Recuperación de contraseña con correo y usuario registrados . . .	33
16.10TC-010: Generación de reporte analítico	34
16.11TC-011: Consulta de historial de reportes por rango de fechas	35
16.12TC-012: Registro de anotación en un reporte existente	36
17.Gestión de Defectos	37
18.Riesgos y Mitigaciones	37
19.Procedimiento de Ejecución	37
20.Entregables	38
21.Conclusiones	38
21.1. Logros Alcanzados	38
21.2. Calidad del Software	38
21.3. Lecciones Aprendidas	38
21.4. Recomendaciones Futuras	39
21.5. Resumen Final	39
22.Conclusiones Grupo 6	39
22.1. Danna Andrade	39
22.2. Ariel Llumiquinga	39
22.3. David Pilaguano	39

Resumen Ejecutivo

Este plan de pruebas presenta la planificación, diseño e implementación de pruebas funcionales y unitarias para el prototipo SNAAR - TekMess. El documento toma como referencia una estructura formal de informe de pruebas: datos generales del proyecto, resumen ejecutivo, objetivos, planteamiento, herramienta utilizada, implementación por requisito, desglose de pruebas, gestión de defectos, riesgos y conclusiones.

El alcance principal se centra en los requisitos funcionales REQ001 a REQ009, que cubren la gestión de empleados, autenticación, cambio y recuperación de contraseña, generación de reportes e historial de reportes. Las pruebas fueron implementadas con JUnit 5 y diseñadas para ejecutarse sobre la lógica de negocio del prototipo, utilizando DAOs en memoria para aislar la capa de persistencia.

Estado de Implementación

Métrica	Valor
Requisitos funcionales evaluados	9/9
Casos de prueba documentados	12 casos detallados
Casos automatizados en JUnit	10 métodos de prueba automatizados
Framework de pruebas	JUnit 5
Tipo de prueba principal	Pruebas funcionales unitarias
Estado del diseño de pruebas	COMPLETADO
Ejecución Maven en el entorno actual	Pendiente por falta de Maven en PATH

Requisitos Funcionales - Estado Final

RF	Nombre	Estado	Cobertura
REQ001	Registro de empleados y generación de credenciales	COMPLETO	Cubierto
REQ002	Listado de personal registrado	COMPLETO	Cubierto
REQ003	Edición de datos del empleado	COMPLETO	Cubierto
REQ004	Eliminación de empleados	COMPLETO	Cubierto
REQ005	Inicio de sesión por rol	COMPLETO	Cubierto
REQ006	Cambio de contraseña	COMPLETO	Cubierto
REQ007	Recuperación de contraseña	COMPLETO	Cubierto
REQ008	Generación de reporte analítico	COMPLETO	Cubierto
REQ009	Consulta de historial de reportes	COMPLETO	Cubierto

1 Control del Documento

1.1 Historial de Versiones

Versión	Fecha	Descripción	Responsable
1.0	09/07/2026	Elaboración inicial del plan de pruebas funcionales, matriz de casos y trazabilidad de requisitos.	Equipo TekMess

1.2 Propósito del Documento

El propósito de este documento es establecer una guía formal para planificar, diseñar y ejecutar las pruebas funcionales del prototipo del sistema SNAAR - TekMess. El plan define los objetivos, alcance, estrategia, ambiente, criterios, datos de prueba, responsabilidades y casos que permiten verificar el cumplimiento de los requisitos funcionales identificados.

Este documento también sirve como evidencia técnica del proceso de aseguramiento de calidad aplicado al prototipo, permitiendo relacionar cada requisito con al menos un caso de prueba automatizado o verificable.

2 Introducción

El sistema SNAAR - TekMess es un prototipo web desarrollado en Java orientado a la gestión de empleados, generación de credenciales, autenticación por rol y administración de reportes analíticos. Debido a que el sistema maneja información de usuarios, roles y accesos, resulta necesario validar que las reglas de negocio se comporten de manera correcta antes de avanzar hacia pruebas integrales con base de datos y servidor de aplicaciones.

Las pruebas propuestas se enfocan en las funcionalidades existentes en el prototipo y se desarrollan con JUnit 5. Para aislar la lógica de negocio y evitar dependencia directa de PostgreSQL durante la ejecución unitaria, se emplean objetos DAO en memoria que simulan el comportamiento esperado de la persistencia.

La planificación considera los requisitos REQ001 a REQ009, relacionados con registro de empleados, listado, edición, eliminación, inicio de sesión, cambio de contraseña, recuperación de contraseña, generación de reportes e historial de reportes. Las pruebas fueron diseñadas desde cero con base en los requisitos entregados, sin reutilizar pruebas previas que pudieran existir en documentos externos.

3 Objetivos

3.1 Objetivo General

Validar de forma sistemática y automatizada que el prototipo TekMess cumpla los requisitos funcionales REQ001 a REQ009, verificando los flujos principales, reglas de validación, estados del sistema y respuestas esperadas en los módulos de empleados, autenticación, contraseñas y reportes.

3.2 Objetivos Específicos

- Verificar que el sistema permita registrar empleados válidos y genere credenciales de acceso iniciales.
- Comprobar que la validación de datos impida registros con cédulas inválidas, roles incorrectos, correos no válidos o datos duplicados.
- Confirmar que el listado de personal presente la información principal de cada empleado registrado.
- Validar la edición de datos de empleados existentes, manteniendo reglas de integridad y consistencia.
- Verificar la eliminación de empleados y la baja del usuario asociado.
- Evaluar el inicio de sesión por rol, el control de intentos fallidos y el bloqueo de cuentas.
- Comprobar el cambio de contraseña con política de seguridad, cifrado y actualización de primer acceso.
- Validar la recuperación de contraseña mediante usuario y correo registrados.
- Verificar la generación de reportes analíticos y la consulta del historial con anotaciones.
- Registrar la trazabilidad entre requisitos, casos de prueba, datos utilizados y resultados esperados.

4 Datos Generales del Proyecto

Campo	Descripción
Nombre del proyecto	Sistema SNAAR - TekMess
Tipo de producto	Prototipo web Java basado en arquitectura MVC, servicios, DAOs y vistas JSP
Lenguaje principal	Java 17
Herramienta de construcción	Maven
Framework de pruebas	JUnit 5
Plugin de ejecución	Maven Surefire

Base de datos prevista	PostgreSQL
Módulos evaluados	Gestión de empleados, autenticación, cambio y recuperación de contraseña, reportes analíticos
Artefactos generados	Pruebas automatizadas JUnit, documento LaTeX del plan de pruebas y tablero Kanban en Excel
Fecha del plan	9 de julio de 2026

5 Involucrados

Integrante	Rol en el proyecto	Responsabilidad en pruebas
Danna Andrade	Integrante del equipo de desarrollo	Revisión de requisitos de gestión de empleados, validación de registro, eliminación y documentación del tablero Kanban.
Ariel Llumiquinga	Integrante del equipo de desarrollo	Revisión de requisitos de autenticación, inicio de sesión, bloqueo por intentos y trazabilidad de casos de prueba.
David Pilaguano	Integrante del equipo de desarrollo	Revisión de requisitos de reportes, historial, anotaciones y consistencia técnica del documento.

6 Planteamiento de las Pruebas

Las pruebas se plantean como un mecanismo de verificación temprana para comprobar que la lógica funcional del prototipo responde a las reglas definidas en los requisitos. Debido a que el proyecto se encuentra en etapa de prototipo, se prioriza la validación de servicios, entidades y utilidades antes de ejecutar pruebas de interfaz o integración completa.

El planteamiento considera tres principios principales:

- **Repetibilidad:** las pruebas deben poder ejecutarse varias veces con los mismos datos y obtener los mismos resultados.
- **Aislamiento:** cada prueba debe controlar sus datos mediante DAOs en memoria para evitar dependencia de una base de datos externa.
- **Trazabilidad:** cada requisito funcional debe relacionarse con pruebas específicas y resultados esperados.

6.1 Enfoque de Testing

El enfoque aplicado corresponde a pruebas funcionales unitarias. Se evalúa el comportamiento observable de los métodos de servicio frente a entradas válidas e inválidas, verificando mensajes, objetos generados, cambios de estado y persistencia simulada.

6.2 Estrategia de Mocking

En lugar de utilizar librerías externas de mocking, se definieron clases simples en memoria que implementan las interfaces DAO del sistema. Esta estrategia resulta suficiente para el prototipo, reduce complejidad y permite controlar de forma explícita los datos de cada escenario.

7 Alcance

7.1 Funcionalidades Incluidas

El plan cubre las siguientes funcionalidades del prototipo:

- Registro de empleados y generación automática de credenciales.
- Listado de empleados registrados.
- Edición de información de empleados.
- Eliminación de empleados y usuario asociado.
- Inicio de sesión por rol y bloqueo por intentos fallidos.
- Cambio de contraseña con política de seguridad.
- Recuperación de contraseña mediante usuario y correo.
- Generación de reportes analíticos.
- Consulta de historial de reportes y registro de anotaciones.

7.2 Funcionalidades No Incluidas

Quedan fuera del alcance de este plan:

- Pruebas de rendimiento, carga o estrés.
- Pruebas de seguridad avanzadas como inyección SQL, análisis de vulnerabilidades o pruebas de penetración.
- Pruebas visuales completas sobre JSP, CSS o experiencia de usuario.

- Pruebas de compatibilidad entre navegadores.
- Pruebas de integración real contra PostgreSQL, salvo que posteriormente se configure un entorno de base de datos controlado.
- Validación de exportación PDF con inspección visual del archivo generado.

8 Estrategia de Pruebas

La estrategia se basa en pruebas funcionales automatizadas sobre la capa de servicios y utilidades. Esta decisión permite validar las reglas de negocio de forma rápida, repetible y controlada, sin depender de elementos externos como servidor web, sesiones HTTP o base de datos real.

Elemento	Estrategia Aplicada
Nivel de prueba	Prueba funcional unitaria sobre servicios Java.
Enfoque	Validación de entradas, reglas de negocio, mensajes de respuesta y cambios de estado.
Aislamiento	Uso de DAOs en memoria para simular empleados, usuarios y reportes.
Automatización	Casos implementados en JUnit 5 dentro de <code>src/test/java</code> .
Datos de prueba	Datos controlados para empleados, usuarios, roles, contraseñas y reportes.
Ejecución	Comando esperado: <code>mvn test</code> .
Evidencia	Resultado de ejecución JUnit, código fuente de pruebas y trazabilidad incluida en este documento.

9 Herramienta

9.1 Framework de Testing

Para la implementación de las pruebas unitarias se utiliza **JUnit 5**, específicamente la dependencia `junit-jupiter`. Este framework permite definir pruebas automatizadas mediante anotaciones, validar resultados con aserciones y ejecutar los casos desde Maven.

La configuración incorporada al archivo `pom.xml` contempla:

- Dependencia `org.junit.jupiter:junit-jupiter:5.10.2`.
- Plugin `maven-surefire-plugin:3.2.5`.
- Compatibilidad con Java 17.

9.2 Entorno de Pruebas

El entorno propuesto para la ejecución formal de pruebas es:

- Java 17 o superior.
- Maven instalado y disponible en PATH.
- Proyecto ubicado en la carpeta raíz donde se encuentra `pom.xml`.
- Comando de ejecución: `mvn test`.

9.3 Estructura de Archivos de Prueba

La implementación de pruebas se ubica dentro de la estructura estándar de Maven:

```
src/  
+-- test/  
    +-- java/  
        +-- com/  
            +-- tekmess/  
                +-- snaar/  
                    +-- RequisitosFuncionalesTest.java
```

El archivo `RequisitosFuncionalesTest.java` concentra los casos automatizados para los requisitos funcionales evaluados. Además, contiene implementaciones DAO en memoria que permiten simular la persistencia de empleados, usuarios y reportes sin requerir una conexión real a PostgreSQL.

9.4 Estrategia de Simulación y Aislamiento

Para evitar que las pruebas dependan de servicios externos, se aplicó una estrategia de aislamiento mediante objetos de prueba:

- `FakeEmpleadoDAO`: simula creación, edición, eliminación, consulta y conteos de empleados.
- `FakeUsuarioDAO`: simula creación de usuarios, búsqueda, actualización de contraseña, intentos fallidos y estado de cuenta.
- `FakeReporteDAO`: simula almacenamiento, consulta histórica y anotaciones de reportes.

Esta estrategia permite validar la lógica de negocio de los servicios sin alterar datos reales y sin depender de la disponibilidad de la base de datos.

10 Ambiente de Pruebas

Recurso	Especificación
Sistema operativo	Windows o entorno compatible con Java y Maven
JDK	Java 17 o superior
Gestor de dependencias	Maven configurado en PATH o Maven Wrapper del proyecto
Framework de pruebas	JUnit Jupiter 5.10.2
Plugin de pruebas	Maven Surefire 3.2.5
Base de datos	No requerida para la ejecución unitaria; se simula mediante DAOs en memoria
Comando de ejecución	<code>mvn test</code>

11 Criterios de Entrada y Salida

11.1 Criterios de Entrada

- El código fuente del prototipo debe estar disponible.
- El archivo `pom.xml` debe incluir JUnit 5 y Maven Surefire.
- Los requisitos funcionales REQ001 a REQ009 deben estar identificados.
- Las clases de servicio y entidades deben compilar correctamente.
- El entorno de ejecución debe contar con Java y Maven.

11.2 Criterios de Salida

- Cada requisito funcional debe tener al menos un caso de prueba asociado.
- Las pruebas automatizadas deben ejecutarse sin errores.
- Los resultados esperados deben coincidir con los resultados obtenidos.
- Las incidencias detectadas deben registrarse para corrección.
- El plan de pruebas y el tablero Kanban deben quedar actualizados como evidencia del trabajo realizado.

12 Datos de Prueba

Dato	Valor de Referencia	Uso Principal
Cédula válida	1712345678	Registro, edición, eliminación y recuperación de usuario.
Nombre de empleado	Danna Andrade	Generación de usuario base dandrade.
Correo institucional	danna@tekness.com	Validación de correo y recuperación de contraseña.
Rol válido	SUPERVISOR, GUARDIA, JE- FE_LOGISTICA	Validación de permisos y creación de sesión por rol.
Contraseña temporal	Temp123!	Inicio de sesión y cambio de contraseña.
Contraseña nueva	Nueva123! / Recupera123!	Cambio y recuperación de contraseña cumpliendo política.
Rango de fechas válido	Fecha inicio menor que fecha fin	Generación y consulta de reportes.

13 Matriz de Trazabilidad

Requisito	Funcionalidad Evaluada	Clase Principal	Casos
REQ001	Registro de empleados y generación de credenciales	EmpleadoServicio, GeneradorCredenciales, ValidadorDatos	TC-001, TC-002
REQ002	Listado de personal registrado	EmpleadoServicio	TC-003
REQ003	Edición de datos del empleado	EmpleadoServicio	TC-004
REQ004	Eliminación de empleados	EmpleadoServicio	TC-005
REQ005	Inicio de sesión por rol	AuthService, Sesion	TC-006, TC-007
REQ006	Cambio de contraseña	AuthService, CifradorContrasena	TC-008
REQ007	Recuperación de contraseña	AuthService	TC-009
REQ008	Generación de reporte analítico	ReporteServicio	TC-010
REQ009	Consulta de historial de reportes	ReporteServicio	TC-011, TC-012

14 Implementación por Requisito

Esta sección describe cómo se implementaron las pruebas para cada requisito funcional. Se detalla el componente evaluado, la lógica cubierta, los escenarios considerados y los resultados esperados. La finalidad es evidenciar no solo la existencia de pruebas, sino también el razonamiento aplicado para cubrir los flujos principales y alternos del sistema.

14.1 REQ001: Registro de empleados y generación de credenciales

Archivos y componentes evaluados:

- **EmpleadoServicio**: creación del empleado, validación de duplicidad y generación de usuario.
- **ValidadorDatos**: validación de cédula, nombres, rol, usuario, correo y contraseña.
- **GeneradorCredenciales**: generación de nombre de usuario y contraseña temporal.
- **CifradorContraseña**: cifrado y verificación de contraseña.

Cómo se implementó: se creó un empleado con datos válidos y se ejecutó el método `crearEmpleado`. La prueba verifica que el empleado sea almacenado, que se genere un usuario asociado a su cédula, que la contraseña temporal cumpla la política de seguridad y que el campo de primer acceso quede activo. También se implementó un escenario de error para rechazar el registro cuando la cédula ya existe.

Aspectos cubiertos:

- Flujo exitoso de creación de empleado.
- Rechazo de cédula duplicada.
- Normalización de correo.
- Generación de usuario base a partir de nombres.
- Generación y cifrado de contraseña temporal.

Resultado esperado: el sistema registra empleados válidos, impide duplicados y genera credenciales seguras para el primer acceso.

14.2 REQ002: Listado de personal registrado

Archivos y componentes evaluados:

- **EmpleadoServicio**: consulta de empleados registrados.
- **IEmpleadoDAO**: operación `listarTodos`.

Cómo se implementó: se registraron empleados en el DAO en memoria y se ejecutó `consultarEmpleados`. La prueba valida que el listado retorne la cantidad esperada de registros y que los campos principales estén disponibles.

Aspectos cubiertos:

- Consulta de todos los empleados registrados.
- Retorno de cédula, nombres, correo y rol.
- Disponibilidad de datos para edición o eliminación posterior.

Resultado esperado: el sistema retorna un listado consistente y completo del personal registrado.

14.3 REQ003: Edición de datos del empleado

Archivos y componentes evaluados:

- `EmpleadoServicio`: edición de empleado.
- `ValidadorDatos`: validación de nuevos datos.
- `IEmpleadoDAO`: búsqueda y actualización por cédula.

Cómo se implementó: se creó un empleado inicial y luego se ejecutó `editarEmpleado` con correo y rol actualizados. La prueba valida que el empleado exista antes de modificarlo y que los cambios se reflejen en el DAO.

Aspectos cubiertos:

- Búsqueda de empleado por cédula.
- Validación de correo y rol.
- Actualización del registro.
- Mensaje de confirmación de edición.

Resultado esperado: los datos modificables del empleado se actualizan correctamente cuando la cédula existe y los datos son válidos.

14.4 REQ004: Eliminación de empleados

Archivos y componentes evaluados:

- `EmpleadoServicio`: eliminación de empleado.
- `IEmpleadoDAO`: búsqueda y eliminación del registro.
- `IUsuarioDAO`: eliminación del usuario asociado.

Cómo se implementó: se registró un empleado y un usuario asociado a la misma cédula. Luego se ejecutó `eliminarEmpleado`. La prueba verifica que ambos registros desaparezcan del almacenamiento en memoria.

Aspectos cubiertos:

- Validación de existencia del empleado.
- Eliminación del usuario asociado.
- Eliminación definitiva del empleado.
- Prevención de accesos futuros para usuarios eliminados.

Resultado esperado: el empleado eliminado deja de existir en el sistema y su cuenta asociada queda removida.

14.5 REQ005: Inicio de sesión por rol

Archivos y componentes evaluados:

- `AuthService`: autenticación y control de intentos.
- `Sesion`: creación de sesión y rol.
- `Usuario`: incremento y reinicio de intentos fallidos.
- `EstadoCuenta`: control de cuenta activa o bloqueada.

Cómo se implementó: se creó un empleado con rol `SUPERVISOR` y un usuario activo con contraseña cifrada. Se validó el inicio de sesión correcto y la creación de sesión con rol. Además, se ejecutaron tres intentos con contraseña incorrecta para comprobar el bloqueo de cuenta.

Aspectos cubiertos:

- Validación de usuario y contraseña.
- Verificación de hash de contraseña.
- Creación de sesión con rol.
- Redirección por primer acceso.
- Bloqueo por intentos fallidos.

Resultado esperado: el sistema permite acceder solo con credenciales correctas, asigna el rol correspondiente y bloquea la cuenta al superar el límite de intentos fallidos.

14.6 REQ006: Cambio de contraseña

Archivos y componentes evaluados:

- **AuthService:** proceso de cambio de contraseña.
- **ValidadorDatos:** política de contraseña.
- **CifradorContrasena:** generación y verificación del hash.

Cómo se implementó: se creó un usuario con contraseña temporal cifrada y primer acceso activo. Luego se ejecutó **cambiarContrasena** con contraseña actual correcta, nueva contraseña y confirmación coincidente.

Aspectos cubiertos:

- Validación de contraseña actual.
- Coincidencia entre nueva contraseña y confirmación.
- Cumplimiento de política de seguridad.
- Rechazo de reutilización de la contraseña actual.
- Cifrado de la nueva contraseña.
- Actualización del indicador de primer acceso.

Resultado esperado: el sistema actualiza la contraseña de forma segura y desactiva el primer acceso obligatorio.

14.7 REQ007: Recuperación de contraseña

Archivos y componentes evaluados:

- **AuthService:** recuperación de contraseña.
- **Empleado:** correo asociado al usuario.
- **Usuario:** cuenta a actualizar.

Cómo se implementó: se creó un empleado con correo institucional y un usuario asociado a su cédula. La prueba ejecuta **recuperarContrasena** con usuario, correo y nueva contraseña válida.

Aspectos cubiertos:

- Validación de usuario.
- Validación de correo.
- Correspondencia entre correo, empleado y usuario.

- Rechazo de cuentas bloqueadas.
- Aplicación de política de contraseña.
- Actualización del hash.

Resultado esperado: la contraseña se restablece únicamente cuando el usuario y correo pertenecen al mismo registro.

14.8 REQ008: Generación de reporte analítico

Archivos y componentes evaluados:

- ReporteServicio: generación del reporte.
- Reporte: entidad con totales consolidados.
- IReporteDAO: almacenamiento del reporte.
- IEmpleadoDAO e IUsuarioDAO: consultas de totales del período.

Cómo se implementó: se configuraron totales simulados para empleados creados, editados, eliminados y accesos fallidos. Luego se ejecutó `generarReporte` con un rango de fechas válido y se verificó el contenido del reporte.

Aspectos cubiertos:

- Validación de fechas.
- Consolidación de empleados creados.
- Consolidación de empleados editados.
- Consolidación de empleados eliminados.
- Consolidación de accesos fallidos.
- Almacenamiento del reporte generado.

Resultado esperado: el sistema genera un reporte con totales correctos y confirma la operación.

14.9 REQ009: Consulta de historial de reportes

Archivos y componentes evaluados:

- ReporteServicio: consulta de historial y anotaciones.
- Reporte: reporte generado previamente.
- Anotacion: observación vinculada al reporte.

- IReporteDAO: búsqueda, listado y registro de anotaciones.

Cómo se implementó: se guardó un reporte en memoria y se consultó el historial con un rango de fechas que lo incluye. Además, se agregó una anotación al reporte y se verificó que quede asociada correctamente.

Aspectos cubiertos:

- Consulta de reportes sin alterar datos base.
- Filtro por rango de fechas.
- Validación de rango de consulta.
- Registro de anotaciones.
- Asociación entre anotación y reporte.

Resultado esperado: el sistema retorna reportes históricos según el filtro aplicado y permite agregar observaciones.

15 Código de Implementación de las Pruebas

Esta sección incorpora fragmentos representativos del código utilizado para implementar las pruebas unitarias. No se incluye la totalidad del archivo fuente para evitar redundancia, pero sí los bloques principales que evidencian la configuración, la inyección de dependencias, la estructura de pruebas y la simulación de persistencia mediante DAOs en memoria.

15.1 Configuración Maven para JUnit 5

La primera implementación necesaria fue agregar JUnit 5 y Maven Surefire al archivo pom.xml. Esta configuración permite que Maven detecte y ejecute automáticamente los métodos anotados con @Test.

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.2</version>
  <scope>test</scope>
</dependency>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.2.5</version>
</plugin>
```

Listing 1: Dependencias y plugin de pruebas en pom.xml

15.2 Ajuste de Testabilidad en AuthService

Para poder probar autenticación y recuperación de contraseña sin consultar directamente la base de datos, se agregó un constructor que recibe `IEmpleadoDAO`. El constructor existente se mantiene, por lo que el comportamiento de producción no se elimina ni se rompe.

```
public class AuthService {

    private final IUsuarioDAO usuarioDAO;
    private final IEmpleadoDAO empleadoDAO;
    private final CifradorContrasena cifrador;
    private final ValidadorDatos validador;

    public AuthService(IUsuarioDAO usuarioDAO) {
        this(usuarioDAO, new EmpleadoDAO());
    }

    public AuthService(IUsuarioDAO usuarioDAO, IEmpleadoDAO empleadoDAO
    ) {
        this.usuarioDAO = usuarioDAO;
        this.empleadoDAO = empleadoDAO;
        this.cifrador = new CifradorContrasena();
        this.validador = new ValidadorDatos();
    }
}
```

Listing 2: Constructor adicional para inyección de dependencias

15.3 Estructura General del Archivo de Pruebas

Las pruebas se concentran en la clase `RequisitosFuncionalesTest`. Cada método se identifica con el requisito funcional evaluado mediante `@DisplayName`, lo que facilita la lectura de resultados durante la ejecución.

```
class RequisitosFuncionalesTest {

    private final CifradorContrasena cifrador = new CifradorContrasena()
    ;

    @Test
    @DisplayName("REQ001 - registra empleado valido y genera
    credenciales")
    void registraEmpleadoGeneraCredenciales() {
        FakeEmpleadoDAO empleadoDAO = new FakeEmpleadoDAO();
        FakeUsuarioDAO usuarioDAO = new FakeUsuarioDAO();
        EmpleadoServicio servicio = new EmpleadoServicio(empleadoDAO,
        usuarioDAO);

        String resultado = servicio.crearEmpleado(
            new Empleado(
                "1712345678",
                "Danna Andrade",
                "DANNA.ANDRADE@TEKMESS.COM",
                Rol.JEFE_LOGISTICA
            )
        );
    }
}
```

```
    );  
  
    assertTrue(resultado.contains("Empleado registrado exitosamente"  
        ));  
    }  
}
```

Listing 3: Estructura base de la clase de pruebas

15.4 Implementación de Prueba para REQ001

El siguiente fragmento valida el flujo principal del registro de empleados: almacenamiento del empleado, creación de usuario, generación de contraseña temporal, cumplimiento de política de seguridad y activación del primer acceso.

```
@Test  
@DisplayName("REQ001 - registra empleado valido y genera credenciales")  
void registraEmpleadoGeneraCredenciales() {  
    FakeEmpleadoDAO empleadoDAO = new FakeEmpleadoDAO();  
    FakeUsuarioDAO usuarioDAO = new FakeUsuarioDAO();  
    EmpleadoServicio servicio = new EmpleadoServicio(empleadoDAO,  
        usuarioDAO);  
  
    String resultado = servicio.crearEmpleado(  
        new Empleado("1712345678", "Danna Andrade",  
            "DANNA.ANDRADE@TEKMESS.COM", Rol.JEFE_LOGISTICA)  
    );  
  
    assertTrue(resultado.contains("Empleado registrado exitosamente"));  
    assertNotNull(empleadoDAO.buscarPorCedula("1712345678"));  
  
    Usuario usuario = usuarioDAO.buscarPorCedula("1712345678");  
    assertNotNull(usuario);  
    assertEquals("dandrade", usuario.getNombreUsuario());  
    assertTrue(usuario.isPrimerAcceso());  
    assertNull(new ValidadorDatos().validarContrasena(  
        usuario.getContrasenaTemporal()  
    ));  
    assertTrue(cifrador.verificar(  
        usuario.getContrasenaTemporal(),  
        usuario.getContrasenaHash()  
    ));  
}
```

Listing 4: Prueba de registro de empleado y generación de credenciales

15.5 Implementación de Prueba para REQ005

Esta prueba cubre dos elementos importantes del inicio de sesión: acceso correcto por rol y bloqueo por intentos fallidos. El escenario valida que el usuario obtenga una sesión con rol SUPERVISOR y que, tras tres contraseñas incorrectas, la cuenta quede bloqueada.

```
@Test  
@DisplayName("REQ005 - inicia sesion, redirige primer acceso y bloquea")
```

```

void inicioSesionPorRolYBloqueo() {
    FakeEmpleadoDAO empleadoDAO = new FakeEmpleadoDAO();
    FakeUsuarioDAO usuarioDAO = new FakeUsuarioDAO();

    empleadoDAO.crear(new Empleado(
        "1712345678", "Danna Andrade",
        "danna@tekless.com", Rol.SUPERVISOR
    ));

    Usuario usuario = new Usuario(
        "1712345678", "dandrade", cifrador.cifrar("Temp123!")
    );
    usuarioDAO.crear(usuario);

    AuthService servicio = new AuthService(usuarioDAO, empleadoDAO);

    Object[] login = servicio.iniciarSesion("dandrade", "Temp123!");
   assertInstanceOf(Sesion.class, login[0]);
    assertEquals("PRIMER_ACCESO", login[1]);
    assertEquals(Rol.SUPERVISOR, ((Sesion) login[0]).getRolUsuario());

    servicio.iniciarSesion("dandrade", "Mala123!");
    servicio.iniciarSesion("dandrade", "Mala123!");
    Object[] bloqueado = servicio.iniciarSesion("dandrade", "Mala123!");

    assertNull(bloqueado[0]);
    assertTrue(((String) bloqueado[1]).contains("Cuenta bloqueada"));
    assertEquals(
        EstadoCuenta.BLOQUEADO,
        usuarioDAO.buscarPorNombreUsuario("dandrade").getEstadoCuenta()
    );
}

```

Listing 5: Prueba de inicio de sesión por rol y bloqueo de cuenta

15.6 Implementación de Prueba para REQ006

El cambio de contraseña se valida comprobando contraseña actual, política de seguridad, cifrado de la nueva clave y desactivación del indicador de primer acceso.

```

@Test
@DisplayName("REQ006 - cambia contraseña validando politica y hash")
void cambiaContraseñaConPolitica() {
    FakeUsuarioDAO usuarioDAO = new FakeUsuarioDAO();

    Usuario usuario = new Usuario(
        "1712345678", "dandrade", cifrador.cifrar("Temp123!")
    );
    usuario.setPrimerAcceso(true);
    usuarioDAO.crear(usuario);

    String resultado = new AuthService(usuarioDAO, new FakeEmpleadoDAO
    ())
        .cambiarContraseña(
            "dandrade",
            "Temp123!",

```

```
        "Nueva123!",  
        "Nueva123!"  
    );  
  
    Usuario actualizado = usuarioDAO.buscarPorNombreUsuario("dandrade");  
    assertTrue(resultado.contains("actualizada exitosamente"));  
    assertTrue(cifrador.verificar(  
        "Nueva123!",  
        actualizado.getContrasenaHash()  
    ));  
    assertFalse(actualizado.isPrimerAcceso());  
}
```

Listing 6: Prueba de cambio de contraseña

15.7 Implementación de Prueba para REQ008

La generación de reportes se evalúa con totales simulados. La prueba confirma que el servicio consolide empleados creados, editados, eliminados y accesos fallidos.

```
@Test  
@DisplayName("REQ008 - genera reporte analitico consolidando datos")  
void generaReporteAnalitico() {  
    FakeEmpleadoDAO empleadoDAO = new FakeEmpleadoDAO();  
    FakeUsuarioDAO usuarioDAO = new FakeUsuarioDAO();  
    FakeReporteDAO reporteDAO = new FakeReporteDAO();  
  
    empleadoDAO.totalCreados = 2;  
    empleadoDAO.totalEditados = 1;  
    empleadoDAO.totalEliminados = 1;  
    usuarioDAO.totalAccesosFallidos = 3;  
  
    Object[] respuesta = new ReporteServicio(  
        reporteDAO, empleadoDAO, usuarioDAO  
    ).generarReporte(new Date(1000), new Date(2000), "Danna Andrade");  
  
   assertInstanceOf(Reporte.class, respuesta[0]);  
    Reporte reporte = (Reporte) respuesta[0];  
  
    assertEquals(2, reporte.getTotalEmpleadosCreados());  
    assertEquals(1, reporte.getTotalEmpleadosEditados());  
    assertEquals(1, reporte.getTotalEmpleadosEliminados());  
    assertEquals(3, reporte.getTotalAccesosFallidos());  
    assertEquals("Reporte generado exitosamente.", respuesta[1]);  
}
```

Listing 7: Prueba de generación de reporte analítico

15.8 Implementación de DAO en Memoria

Los DAOs en memoria reemplazan temporalmente la persistencia real. Esto permite ejecutar pruebas controladas sin base de datos. El siguiente ejemplo muestra la lógica principal del DAO de empleados usado en pruebas.

```
private static class FakeEmpleadoDAO implements IEmpleadoDAO {

    private final Map<String, Empleado> empleados = new HashMap<>();
    int totalCreados;
    int totalEditados;
    int totalEliminados;

    @Override
    public boolean crear(Empleado empleado) {
        empleados.put(empleado.getCedula(), empleado);
        totalCreados++;
        return true;
    }

    @Override
    public boolean editar(Empleado empleado) {
        if (!empleados.containsKey(empleado.getCedula())) {
            return false;
        }
        empleados.put(empleado.getCedula(), empleado);
        totalEditados++;
        return true;
    }

    @Override
    public boolean eliminar(String cedula) {
        Empleado eliminado = empleados.remove(cedula);
        if (eliminado != null) {
            totalEliminados++;
            return true;
        }
        return false;
    }

    @Override
    public Empleado buscarPorCedula(String cedula) {
        return empleados.get(cedula);
    }

    @Override
    public List<Empleado> listarTodos() {
        return new ArrayList<>(empleados.values());
    }
}
```

Listing 8: DAO en memoria para empleados

15.9 Comando de Ejecución

Una vez instalado Maven en el entorno, la ejecución completa de las pruebas se realiza desde la raíz del proyecto mediante:

```
mvn test
```

Listing 9: Comando para ejecutar pruebas

16 Desglose Detallado de Pruebas Funcionales

En esta sección se describe cada prueba de forma individual. Para cada caso se especifica el objetivo, requisito funcional relacionado, precondiciones, datos utilizados, procedimiento, elementos evaluados, resultado esperado y criterio de aprobación. Esta estructura permite comprender exactamente qué se valida en cada funcionalidad sin reducir el análisis a una matriz resumida.

16.1 TC-001: Registro correcto de empleado y generación de credenciales

Requisito asociado: REQ001 - Registro de empleados y generación de credenciales.

Objetivo de la prueba: comprobar que el sistema permita registrar un empleado con datos válidos y que, como consecuencia del registro, genere automáticamente una cuenta de usuario con credenciales temporales.

Precondiciones:

- No debe existir un empleado registrado con la cédula utilizada en la prueba.
- El DAO de empleados debe estar disponible en memoria.
- El DAO de usuarios debe estar disponible en memoria.

Datos de prueba:

- Cédula: 1712345678.
- Nombres: Danna Andrade.
- Correo: DANNA.ANDRADE@TEKMESS.COM.
- Rol: JEFE_LOGISTICA.

Procedimiento:

1. Crear una instancia de `EmpleadoServicio` con DAOs en memoria.
2. Construir un objeto `Empleado` con datos válidos.
3. Ejecutar el método `crearEmpleado`.
4. Consultar el empleado almacenado por cédula.
5. Consultar el usuario generado para la cédula registrada.

Aspectos evaluados:

- Validación de cédula con exactamente diez dígitos numéricos.

- Validación de nombres completos.
- Validación de rol permitido por el sistema.
- Normalización del correo institucional a minúsculas.
- Generación de nombre de usuario a partir de nombres y apellidos.
- Generación de contraseña temporal que cumpla política de seguridad.
- Cifrado de la contraseña mediante BCrypt.
- Marcación del usuario con primer acceso obligatorio.

Resultado esperado: el servicio debe retornar un mensaje de registro exitoso, el empleado debe existir en el DAO, el usuario debe estar asociado a la cédula del empleado, la contraseña temporal debe cumplir la política de seguridad y el campo de primer acceso debe quedar activo.

Criterio de aprobación: la prueba se aprueba si todas las validaciones anteriores se cumplen y la contraseña temporal coincide con el hash almacenado al verificarla con el cifrador.

16.2 TC-002: Rechazo de registro con cédula duplicada

Requisito asociado: REQ001 - Registro de empleados y generación de credenciales.

Objetivo de la prueba: verificar que el sistema impida registrar un nuevo empleado cuando la cédula ya existe en el repositorio de empleados.

Precondiciones:

- Debe existir previamente un empleado registrado con la misma cédula.
- El servicio debe consultar la existencia de la cédula antes de crear el nuevo registro.

Datos de prueba:

- Cédula duplicada: 1712345678.
- Primer empleado: Danna Andrade.
- Segundo empleado: Ariel Llumiquinga.

Procedimiento:

1. Registrar un empleado inicial en el DAO en memoria.
2. Intentar registrar un segundo empleado usando la misma cédula.
3. Capturar el mensaje retornado por `crearEmpleado`.

Aspectos evaluados:

- Verificación de unicidad de la cédula.
- Prevención de duplicidad en el registro de personal.
- Control de flujo para evitar creación de usuario cuando el empleado no debe registrarse.
- Mensaje funcional comprensible para el usuario.

Resultado esperado: el sistema debe rechazar el registro y retornar un mensaje indicando que la cédula ya se encuentra registrada.

Criterio de aprobación: la prueba se aprueba si no se crea un nuevo empleado, no se generan credenciales adicionales y el mensaje de error corresponde al escenario de duplicidad.

16.3 TC-003: Listado de personal registrado

Requisito asociado: REQ002 - Listado de personal registrado.

Objetivo de la prueba: confirmar que el servicio permita consultar el conjunto de empleados registrados y que la información principal de cada empleado esté disponible para la vista de listado.

Precondiciones:

- Deben existir empleados cargados en el DAO en memoria.
- Cada empleado debe tener cédula, nombres, correo y rol.

Datos de prueba:

- Empleado 1: Danna Andrade, rol JEFE_LOGISTICA.
- Empleado 2: Ariel Llumiquinga, rol SUPERVISOR.

Procedimiento:

1. Registrar dos empleados en el DAO en memoria.
2. Ejecutar el método `consultarEmpleados`.
3. Verificar la cantidad de elementos retornados.
4. Validar que uno de los empleados contenga cédula, rol y correo esperados.

Aspectos evaluados:

- Recuperación del listado completo de empleados.
- Conservación de datos principales del empleado.
- Disponibilidad de información necesaria para edición o eliminación posterior.

Resultado esperado: el método debe retornar una lista con los empleados registrados y sus datos principales.

Criterio de aprobación: la prueba se aprueba si la lista contiene la cantidad esperada de empleados y los datos consultados coinciden con los registros cargados.

16.4 TC-004: Edición de datos de un empleado existente

Requisito asociado: REQ003 - Edición de datos del empleado.

Objetivo de la prueba: validar que el sistema permita modificar datos editables de un empleado existente, conservando la cédula como identificador principal.

Precondiciones:

- Debe existir un empleado registrado con la cédula indicada.
- Los nuevos datos deben cumplir las reglas de validación.

Datos de prueba:

- Cédula: 1712345678.
- Correo inicial: `danna@tekmess.com`.
- Correo actualizado: `danna.andrade@tekmess.com`.
- Rol actualizado: SUPERVISOR.

Procedimiento:

1. Crear un empleado inicial en el DAO.
2. Construir un objeto `Empleado` con la misma cédula y datos modificados.
3. Ejecutar el método `editarEmpleado`.
4. Consultar el empleado por cédula.
5. Comparar los datos actualizados.

Aspectos evaluados:

- Búsqueda de empleado existente por cédula.
- Validación de correo institucional.
- Validación de rol permitido.
- Persistencia de cambios en el repositorio.
- Mensaje de confirmación de actualización.

Resultado esperado: el empleado debe quedar actualizado con el nuevo correo y rol, y el servicio debe informar que los datos fueron actualizados exitosamente.

Criterio de aprobación: la prueba se aprueba si el registro consultado después de la edición refleja los cambios enviados y el contador de ediciones del DAO en memoria aumenta.

16.5 TC-005: Eliminación de empleado y usuario asociado

Requisito asociado: REQ004 - Eliminación de empleados.

Objetivo de la prueba: verificar que al eliminar un empleado también se elimine o invalide la cuenta de usuario asociada a su cédula.

Precondiciones:

- Debe existir un empleado registrado.
- Debe existir un usuario asociado a la misma cédula.

Datos de prueba:

- Cédula: 1712345678.
- Usuario asociado: dandrade.
- Contraseña temporal inicial: Temp123!.

Procedimiento:

1. Registrar un empleado en el DAO de empleados.
2. Registrar un usuario asociado en el DAO de usuarios.
3. Ejecutar el método `eliminarEmpleado`.
4. Consultar nuevamente el empleado por cédula.
5. Consultar nuevamente el usuario por cédula.

Aspectos evaluados:

- Localización del empleado antes de eliminar.
- Eliminación del usuario asociado.
- Eliminación del registro de empleado.
- Mensaje de eliminación exitosa.
- Prevención de accesos posteriores con usuario de empleado eliminado.

Resultado esperado: el empleado y el usuario asociado no deben existir después de ejecutar la eliminación.

Criterio de aprobación: la prueba se aprueba si ambas consultas posteriores retornan null y el servicio informa eliminación exitosa.

16.6 TC-006: Inicio de sesión válido por rol

Requisito asociado: REQ005 - Inicio de sesión por rol.

Objetivo de la prueba: comprobar que un usuario activo con credenciales correctas pueda iniciar sesión y que el sistema construya una sesión con el rol correspondiente.

Precondiciones:

- Debe existir un empleado con rol definido.
- Debe existir un usuario activo vinculado a la cédula del empleado.
- La contraseña ingresada debe coincidir con el hash almacenado.

Datos de prueba:

- Usuario: dandrade.
- Contraseña: Temp123!.
- Rol esperado: SUPERVISOR.

Procedimiento:

1. Crear un empleado con rol SUPERVISOR.
2. Crear un usuario activo asociado al empleado.
3. Ejecutar `iniciarSesion` con usuario y contraseña correctos.
4. Verificar que el objeto retornado sea una instancia de `Sesion`.
5. Verificar el rol asignado a la sesión.

Aspectos evaluados:

- Validación de formato del nombre de usuario.
- Verificación de contraseña cifrada.
- Revisión de estado de cuenta activo.
- Reinicio de intentos fallidos tras autenticación correcta.
- Creación de sesión con rol funcional.
- Redirección a cambio de contraseña cuando el usuario está en primer acceso.

Resultado esperado: el sistema debe retornar una sesión válida con rol SUPERVISOR y el mensaje PRIMER_ACCESO cuando corresponda.

Criterio de aprobación: la prueba se aprueba si la sesión no es nula, el rol coincide con el empleado asociado y la respuesta funcional corresponde al estado de primer acceso.

16.7 TC-007: Bloqueo de cuenta por intentos fallidos

Requisito asociado: REQ005 - Inicio de sesión por rol.

Objetivo de la prueba: verificar que el sistema controle los intentos fallidos de autenticación y bloquee la cuenta al alcanzar el límite permitido.

Precondiciones:

- Debe existir un usuario activo.
- La cuenta no debe estar bloqueada al inicio de la prueba.

Datos de prueba:

- Usuario: dandrade.
- Contraseña incorrecta: Mala123!.
- Límite de intentos: 3.

Procedimiento:

1. Ejecutar `iniciarSesion` con contraseña incorrecta.
2. Repetir la operación hasta completar tres intentos fallidos.
3. Consultar el estado de la cuenta.
4. Revisar el mensaje funcional retornado por el tercer intento.

Aspectos evaluados:

- Incremento del contador de intentos fallidos.
- Persistencia del número de intentos.
- Cambio de estado de cuenta a BLOQUEADO.
- Respuesta funcional ante cuenta bloqueada.
- Prevención de acceso con credenciales incorrectas.

Resultado esperado: luego del tercer intento fallido, el usuario debe quedar con estado BLOQUEADO y el sistema debe retornar un mensaje de bloqueo.

Criterio de aprobación: la prueba se aprueba si no se crea sesión, el estado de la cuenta cambia a bloqueado y el mensaje informa que se debe contactar al administrador.

16.8 TC-008: Cambio de contraseña válido

Requisito asociado: REQ006 - Cambio de contraseña.

Objetivo de la prueba: comprobar que un usuario pueda cambiar su contraseña cuando ingresa correctamente la clave actual y la nueva contraseña cumple la política de seguridad.

Precondiciones:

- Debe existir un usuario registrado.
- La contraseña actual debe estar cifrada.
- El usuario debe estar marcado inicialmente con primer acceso obligatorio.

Datos de prueba:

- Usuario: dandrade.
- Contraseña actual: Temp123!.
- Nueva contraseña: Nueva123!.
- Confirmación: Nueva123!.

Procedimiento:

1. Crear un usuario con contraseña temporal cifrada.
2. Ejecutar `cambiarContrasena`.
3. Consultar el usuario actualizado.
4. Verificar que el nuevo hash corresponda a la nueva contraseña.
5. Verificar que el indicador de primer acceso quede desactivado.

Aspectos evaluados:

- Obligatoriedad de contraseña actual.
- Coincidencia entre nueva contraseña y confirmación.
- Cumplimiento de política: longitud mínima, mayúscula, minúscula, dos números y carácter especial.
- Rechazo de contraseña igual a la actual.
- Actualización segura mediante hash.
- Desactivación del primer acceso obligatorio.

Resultado esperado: el sistema debe actualizar la contraseña, almacenar un nuevo hash y retornar un mensaje de actualización exitosa.

Criterio de aprobación: la prueba se aprueba si el nuevo hash valida la contraseña nueva y el usuario deja de estar marcado como primer acceso.

16.9 TC-009: Recuperación de contraseña con correo y usuario registrados

Requisito asociado: REQ007 - Recuperación de contraseña.

Objetivo de la prueba: validar que un usuario pueda restablecer su contraseña cuando proporciona un nombre de usuario y correo electrónico registrados en el sistema.

Precondiciones:

- Debe existir un empleado con correo registrado.
- Debe existir un usuario asociado a la cédula del empleado.
- La cuenta no debe estar bloqueada.

Datos de prueba:

- Correo: `danna@tekmess.com`.
- Usuario: `dandrade`.
- Nueva contraseña: `Recupera123!`.
- Confirmación: `Recupera123!`.

Procedimiento:

1. Registrar empleado y usuario asociado.
2. Ejecutar `recuperarContrasena`.
3. Consultar el usuario por nombre de usuario.
4. Verificar que el hash corresponda a la nueva contraseña.

Aspectos evaluados:

- Validación de formato de usuario.
- Validación de formato de correo.
- Verificación de relación entre usuario, cédula y correo del empleado.
- Bloqueo del restablecimiento cuando la cuenta está bloqueada.
- Aplicación de la política de seguridad a la nueva contraseña.
- Almacenamiento cifrado de la contraseña restablecida.

Resultado esperado: el sistema debe restablecer la contraseña y confirmar la operación mediante un mensaje exitoso.

Criterio de aprobación: la prueba se aprueba si el hash almacenado valida la nueva contraseña y el mensaje indica restablecimiento exitoso.

16.10 TC-010: Generación de reporte analítico

Requisito asociado: REQ008 - Generación de reporte analítico.

Objetivo de la prueba: verificar que el servicio consolide correctamente los datos de empleados gestionados y accesos fallidos en un reporte analítico.

Precondiciones:

- Debe existir un rango de fechas válido.
- Los DAOs deben proporcionar totales de empleados creados, editados, eliminados y accesos fallidos.
- Debe existir un DAO de reportes capaz de guardar el reporte generado.

Datos de prueba:

- Empleados creados: 2.
- Empleados editados: 1.
- Empleados eliminados: 1.
- Accesos fallidos: 3.
- Generado por: Danna Andrade.

Procedimiento:

1. Configurar los totales en los DAOs en memoria.
2. Ejecutar `generarReporte` con fecha inicial menor que fecha final.
3. Obtener el objeto `Reporte` retornado.
4. Validar cada total consolidado.
5. Confirmar el mensaje de generación exitosa.

Aspectos evaluados:

- Validación de rango de fechas.
- Consulta de empleados creados en el período.
- Consulta de empleados editados en el período.
- Consulta de empleados eliminados en el período.
- Consulta de accesos fallidos.
- Construcción del objeto `Reporte`.

- Persistencia del reporte generado.

Resultado esperado: el servicio debe retornar un reporte con los totales esperados y el mensaje `Reporte generado exitosamente`.

Criterio de aprobación: la prueba se aprueba si todos los totales del reporte coinciden con los datos configurados y el reporte queda guardado.

16.11 TC-011: Consulta de historial de reportes por rango de fechas

Requisito asociado: REQ009 - Consulta de historial de reportes.

Objetivo de la prueba: comprobar que el sistema permita consultar reportes históricos aplicando un filtro de fechas válido.

Precondiciones:

- Debe existir al menos un reporte guardado.
- El rango de consulta debe incluir el período del reporte.

Datos de prueba:

- Fecha inicial de reporte: 1000 ms.
- Fecha final de reporte: 2000 ms.
- Rango de búsqueda: 500 ms a 3000 ms.
- Generado por: David Pilaguano.

Procedimiento:

1. Crear y guardar un reporte en el DAO en memoria.
2. Ejecutar `consultarHistorial` con un rango válido.
3. Verificar que el resultado no sea nulo.
4. Validar que la lista contenga el reporte esperado.

Aspectos evaluados:

- Validación de rango de fechas para consulta.
- Recuperación de reportes históricos.
- Aplicación del filtro por período.
- Mensaje funcional OK cuando existen resultados.

Resultado esperado: el sistema debe retornar una lista con el reporte ubicado dentro del rango de fechas y el estado de respuesta debe ser OK.

Criterio de aprobación: la prueba se aprueba si el historial contiene exactamente el reporte esperado y la respuesta funcional indica éxito.

16.12 TC-012: Registro de anotación en un reporte existente

Requisito asociado: REQ009 - Consulta de historial de reportes.

Objetivo de la prueba: validar que se puedan agregar anotaciones a un reporte previamente generado sin modificar los datos base del reporte.

Precondiciones:

- Debe existir un reporte guardado.
- El identificador del reporte debe ser válido.

Datos de prueba:

- Autor: Ariel Llumiquinga.
- Contenido de la anotación: Revision aprobada.

Procedimiento:

1. Guardar un reporte en el DAO en memoria.
2. Ejecutar `agregarAnotacion` indicando el identificador del reporte.
3. Consultar las anotaciones asociadas al reporte.
4. Verificar que la anotación haya sido almacenada.

Aspectos evaluados:

- Búsqueda del reporte por identificador.
- Creación de objeto `Anotacion`.
- Asociación de la anotación al reporte correcto.
- Conservación de los datos base del reporte.
- Respuesta booleana de éxito.

Resultado esperado: la anotación debe quedar registrada y asociada al reporte existente.

Criterio de aprobación: la prueba se aprueba si el método retorna `true` y la consulta de anotaciones devuelve el registro agregado.

17 Gestión de Defectos

Cuando una prueba falle, se deberá registrar una incidencia con la siguiente información mínima:

- Identificador del defecto.
- Caso de prueba asociado.
- Requisito funcional afectado.
- Descripción clara del comportamiento observado.
- Resultado esperado.
- Evidencia de ejecución.
- Severidad: crítica, alta, media o baja.
- Estado: abierto, en corrección, corregido, verificado o cerrado.

18 Riesgos y Mitigaciones

Riesgo	Impacto	Mitigación
Maven no disponible en el entorno	No se pueden ejecutar las pruebas con <code>mvn test</code> .	Instalar Maven, configurar PATH o agregar Maven Wrapper al proyecto.
Dependencia de base de datos real en algunas clases	Puede dificultar pruebas unitarias puras.	Injectar interfaces DAO y usar dobles de prueba en memoria.
Reglas funcionales incompletas en el prototipo	Algunos escenarios podrían requerir validación manual o integración futura.	Documentar limitaciones y ampliar pruebas conforme evolucione el código.
Codificación incorrecta de caracteres	Puede afectar documentación y mensajes en español.	Guardar archivos en UTF-8 y revisar salida del compilador LaTeX.

19 Procedimiento de Ejecución

1. Verificar que Java esté instalado mediante `java -version`.
2. Verificar que Maven esté instalado mediante `mvn -version`.
3. Ubicarse en la raíz del prototipo, donde se encuentra el archivo `pom.xml`.

4. Ejecutar el comando `mvn test`.
5. Revisar el resumen de ejecución generado por Maven Surefire.
6. En caso de fallos, identificar el caso afectado y registrar la incidencia correspondiente.

20 Entregables

Entregable	Descripción
Archivo de pruebas JUnit	Clase <code>RequisitosFuncionalesTest.java</code> con casos automatizados para REQ001 a REQ009.
Plan de pruebas en LaTeX	Documento <code>Plan_Pruebas_TekMess.tex</code> con planificación, estrategia y matriz de casos.
Tablero Kanban en Excel	Archivo <code>Tablero_Kanban_Pruebas_TekMess.xlsx</code> con tareas ejecutadas y responsables.

21 Conclusiones

21.1 Logros Alcanzados

Se diseñó un plan de pruebas funcionales y unitarias alineado con los requisitos REQ001 a REQ009 del prototipo SNAAR - TekMess. Cada requisito cuenta con pruebas documentadas y con una explicación detallada de los elementos evaluados, datos empleados, pasos de ejecución y criterios de aprobación.

También se incorporó la configuración necesaria para ejecutar pruebas con JUnit 5 desde Maven, incluyendo la dependencia `junit-jupiter` y el plugin `maven-surefire-plugin`. Además, se ajustó la clase `AuthService` para permitir la inyección de dependencias y facilitar pruebas sin conexión real a base de datos.

21.2 Calidad del Software

El diseño de pruebas contribuye a mejorar la calidad del prototipo porque valida reglas críticas relacionadas con seguridad, autenticación, credenciales, cambios de contraseña y trazabilidad de reportes. Estas funcionalidades son sensibles porque afectan directamente el acceso al sistema y la confiabilidad de la información gestionada.

21.3 Lecciones Aprendidas

El uso de DAOs en memoria permite aislar la lógica de negocio y construir pruebas rápidas, controladas y repetibles. Asimismo, se evidenció la importancia de diseñar servicios con dependencias inyectables, ya que esto facilita la automatización de pruebas y reduce el acoplamiento con implementaciones concretas.

21.4 Recomendaciones Futuras

Para una siguiente etapa se recomienda:

- Agregar pruebas de integración contra una base PostgreSQL de pruebas.
- Incorporar pruebas de controladores HTTP y filtros de autorización.
- Generar reportes de cobertura con herramientas como JaCoCo.
- Automatizar la ejecución de `mvn test` en un flujo de integración continua.
- Complementar las pruebas unitarias con pruebas funcionales de interfaz para las vistas JSP.

21.5 Resumen Final

El plan de pruebas permite demostrar que el prototipo cuenta con una base sólida de validación funcional. Aunque la ejecución completa depende de contar con Maven instalado en el entorno, los casos diseñados y documentados proporcionan una guía clara para verificar el cumplimiento de los requisitos antes de la entrega final.

22 Conclusiones Grupo 6

■ 22.1 Danna Andrade

El uso de la IA generativa en el desarrollo del plan de pruebas me ayudó a organizar de mejor manera los requisitos funcionales del sistema y transformarlos en casos de prueba claros, completos y verificables. Además, permitió identificar escenarios de éxito y error que fortalecen la validación del prototipo antes de su entrega final.

■ 22.2 Ariel Llumiquinga

La IA generativa en el desarrollo del plan de pruebas me ayudó a comprender con mayor detalle cómo relacionar cada requisito funcional con pruebas unitarias en Java usando JUnit. También facilitó la documentación técnica del proceso, permitiendo describir objetivos, datos de prueba, resultados esperados y criterios de aprobación de forma más profesional.

■ 22.3 David Pilaguano

La implementación de IA generativa en el desarrollo del plan de pruebas me ayudó a estructurar el documento de manera más completa, integrando la planificación, la implementación de pruebas, el código utilizado y las conclusiones del trabajo. Gracias a esto, me fue posible generar un informe más ordenado y alineado con el formato solicitado para el proyecto.